

BUILD YOUR AGENTIC WORKFLOW

MARKDOWN-DRIVEN AI FOR REAL PROJECTS

React Alicante, 24.09.26 26

ABOUT ME



Founder & CTO at Philomath Academy

Previously:

Elastic, IAM, Cloud

TechnoPark Zurich, ETH Spin-off, Auto ML

Google (contractor) Google Assistant

Swiss Learning Hub: LMS Tracker - Time2Learn

St Georges, London: Knowledge Graphs, Med Education

Goal

Build tooling that speeds up our software development cycle.

- Plan, build, release
- Today: two tickets with an agent, then a release to Production
- Everything is markdown in your repo

Principles

- AI executes our engineering judgement
- No vibe-engineering: one prompt, then wait for full execution
- We write our experience into tooling, to enforce our standards
- Guardrails are enforced by tooling
- Human-in-the-loop breakpoints at critical steps

Tooling Landscape in Claude Code

Rules: always-on conventions (4 in the starter)

Skills: one job, done the same way every time (27)

Agents: multi-step workflows that stop for you (7)

AGENTS.md / CLAUDE.md: project onboarding

Hooks: scripts that run on events (not used today)

Agents vs Skills

Does it need to remember, and does a human need to approve in the middle?

SKILL

Remember? No · Approve? No

- Atomic: one bounded job
- Does the work: the “how”
- No memory: run it twice, same result
- Steps are fixed when it is written

Example: release-bump-version

Sets package.json to 0.4.0 and commits it.

AGENT

Remember? Yes · Approve? Yes

- Orchestrates: the “what” and “when”
- Remembers: keeps a state file
- Stops for you between phases
- Picks the next step while it runs

Example: release-manager

Runs release-create-branch, release-bump-version, release-generate-changelog... Stops after each.

The agent decides what and when. The skill does the how.
Make it an agent only when you need memory or a stop.

Rules are different: always on, nothing to start.
Example: layers.md

Anatomy of a skill

A real file: `.claude/skills/release-bump-version/SKILL.md`

`name: release-bump-version`

`description: Bump the version...`

`trigger: manual after: [...]`

`## When to use`

`## Inputs ## Prerequisites`

`## Workflow`

`## Constraints`

`## Output ## Verification`

Name

You start it with `/release-bump-version`

Description

What it does. Claude reads it to know when to use it

Metadata

How it starts, and which skill runs before it

When it is called

The situations that call this skill

Before it starts

What you give it. Checks before it starts: if one fails, it stops

The work

The numbered steps: the “how”

Guardrails

What it must not do

The result

What you get, and how to check that it worked

Anatomy of an agent

A real file: `.claude/agents/release-manager.md`

`name / description / model`

`# Role`

`## Input    ## Prerequisites`

`## State File`

`## Phases + breakpoints`

`## Skills Used`

`## Constraints    ## Failure Handling`

`## Boundaries`

Header

Who it is, and which model runs it

Role

One sentence: which workflow it runs

Before it starts

What you give it. Checks before it starts

Memory

What it remembers between phases and sessions

The workflow

The “what” and “when”. Stops for you after each phase

Its skills

The skills it calls for the “how”

Guardrails

The rules, and what to do when a step fails

Limits

The hard edges: what it will never do

Constant Refinement

We detect code smells and in every iteration we feed back the skills.

What needs a hard-constraint and how such a constraint technically be imposed?

How is the DX with the engineer? Are messages too verbose or too short. Are CTAs clear?

How can we improve accuracy? Verification pass by another agent. Currently it does on its own self during code

- Workflows run as a script Claude writes, executed by the runtime — can fan out to dozens/hundreds of subagents in parallel. Your agent coordinates one or two subagents at a time, turn by turn.
- Workflows have no mid-run human checkpoints (only permission/usage-limit pauses). Your breakpoints (wait for explicit confirmation between phases) don't have an equivalent there.
- Workflows keep state in script variables outside context; yours uses the state file + execution order.
- Workflows save as reusable /commands; yours is a hand-authored .claude/agents/feature-builder.md.

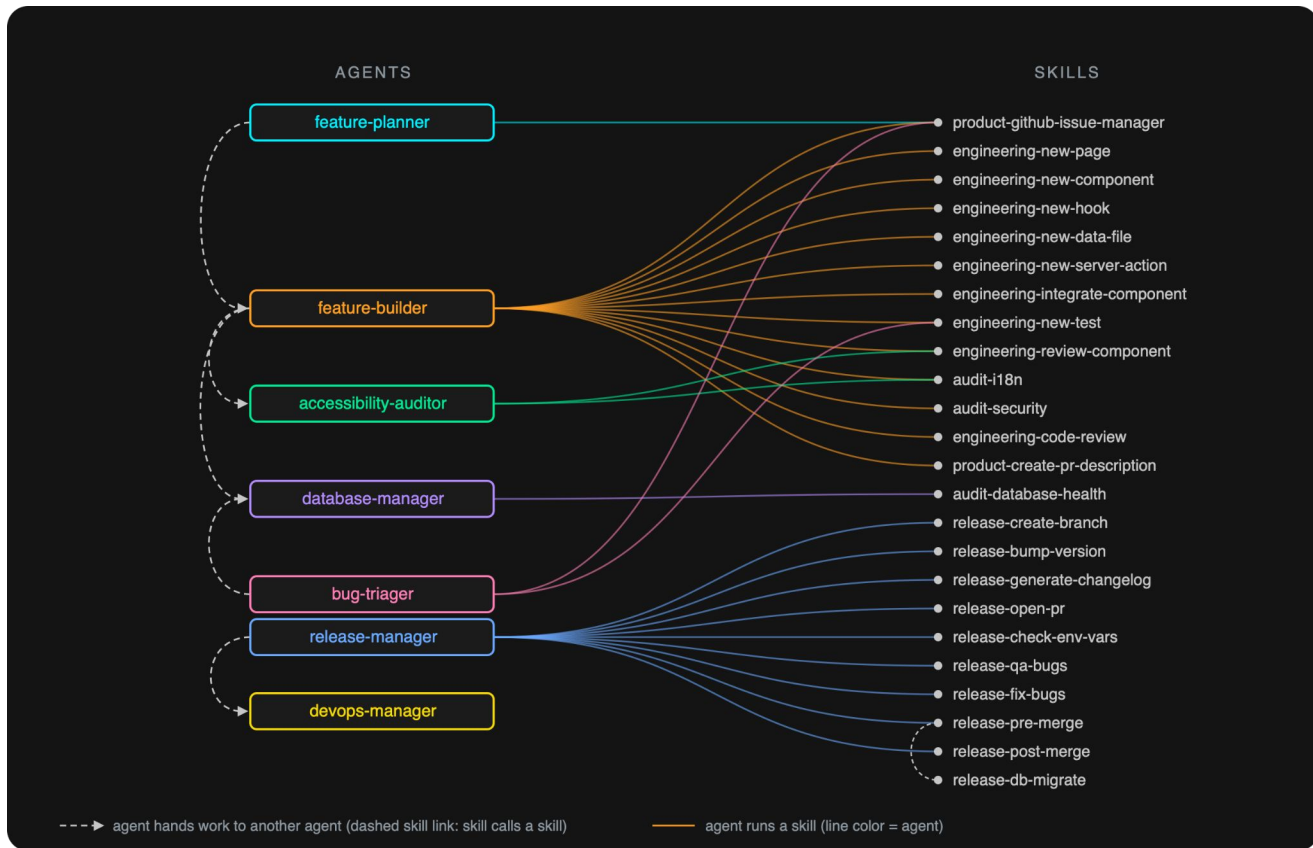
Make the skills yours: adjust them to your stack - 1

Part	Our skills assume	Change it to your...	Skills to edit
Test runner	Jest + RTL	Vitest, ...	<code>engineering-new-test</code> , <code>engineering-new-component</code> , <code>audit-test-coverage</code>
UI library	Chakra UI v3	<code>shadcn/ui</code> + Tailwind, MUI	<code>engineering-new-component</code> , <code>engineering-review-component</code> , <code>engineering-new-page</code>
Styling	CSS variables + SCSS	Tailwind only, CSS-in-JS	<code>engineering-new-component</code> , <code>engineering-review-component</code>
Framework	Next.js App Router	Pages Router, Vite SPA, Remix	<code>most engineering-*</code> , <code>audit-security</code> , <code>audit-i18n</code>

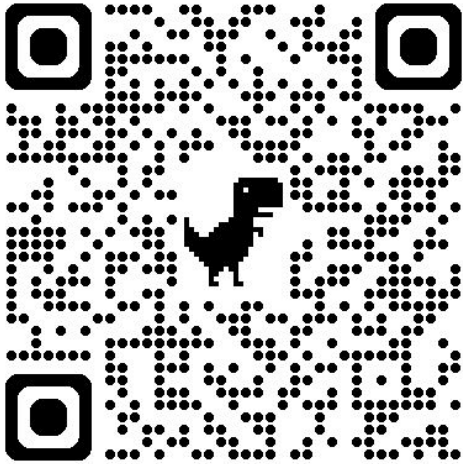
Make the skills yours: adjust them to your stack - 2

Part	Our skills assume	Change it to your...	Skills to edit
Forms	react-hook-form + Zod	Formik + Yup	engineering-new-component, engineering-new-server-action, audit-security
i18n	next-intl	react-i18next, none	audit-i18n, engineering-new-component
Database	Supabase	Prisma, Firebase	audit-database-health, audit-security, release-db-migrate
Logging	pino + Sentry	none, Datadog	engineering-new-server-action, audit-security, release-qa-bugs
CI / deploy	GitHub Actions + Vercel	GitLab, Netlify	release-*, product-create-pr-description

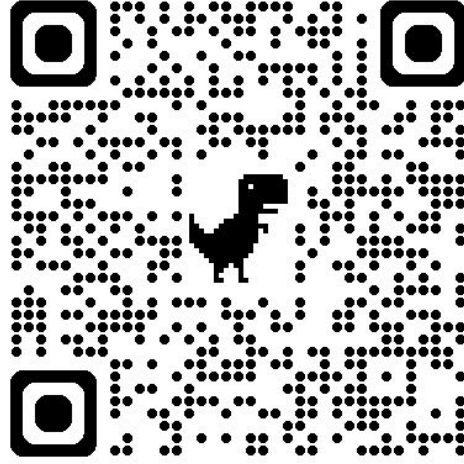
Skills and Agents Graph



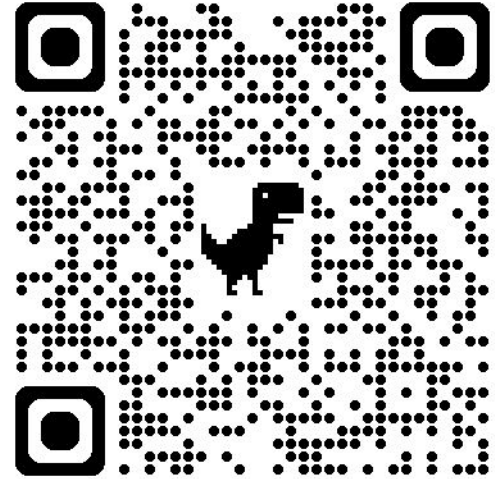
Thank you



Philomath Academy



Github Starter Repo



Follow me on LinkedIn